

A COOKBOOK FOR DEVELOPERS

Applied Data Analysis Cookbook

Turn raw data into answers — pandas, analytical SQL, cleaning, statistics, and visualization — plus using an LLM to do the analysis with you.

pandas

NumPy

SQL

EDA

statistics

matplotlib / seaborn

time series

LLM analysis

Written for a developer who knows Python and wants the analysis foundation under ML and AI — the craft of getting insight from data.

Every chapter answers: what it's for, how to do it in Python/SQL, the pitfalls, and when a simpler tool wins.

Edition 2026 · question → data → answer → decision

Table of Contents

How to Use This Book	4
The one idea: question → data → answer → decision	4
What this book covers	5
Part 1 • The Analyst's Workflow	6
1.1 The four kinds of analysis	6
1.2 The pipeline, step by step	6
1.3 Frame the question first	6
Part 2 • pandas — Your Core Tool	8
2.1 DataFrame and Series	8
2.2 Selecting and filtering	8
2.3 Creating and transforming columns	9
2.4 Vectorise — don't loop	9
Part 3 • Analytical SQL	10
3.1 Aggregation is the backbone	10
3.2 Window functions — the analyst's superpower	10
3.3 CTEs for readable, staged queries	11
3.4 Push work down to the database	11
Part 4 • Cleaning Messy Data	13
4.1 Find the problems first	13
4.2 Missing values	13
4.3 Types, duplicates, and inconsistent categories	13
4.4 Outliers — investigate, don't reflexively delete	14
Part 5 • Reshaping: Group-By, Pivot & Join	15
5.1 Group-by: split-apply-combine	15
5.2 Pivot: long ↔ wide	15
5.3 Join: combining datasets	16
Part 6 • Exploratory Data Analysis (EDA)	17
6.1 Look at one variable at a time (univariate)	17
6.2 Look at relationships (bivariate)	17
6.3 Correlation is a hint, not a conclusion	17
6.4 Segment — the averages lie	18
Part 7 • The Statistics That Keep You Honest	19
7.1 Sampling and uncertainty	19
7.2 Hypothesis testing and the p-value	19
7.3 Correlation vs causation	20
7.4 A/B tests, done honestly	20
Part 8 • Visualization	21
8.1 The tools	21
8.2 Match the chart to the question	21
8.3 Don't mislead (even accidentally)	22

Part 9 · Time-Series Analysis.	23
9.1 A datetime index unlocks pandas' time tools	23
9.2 Resampling — change the time grain	23
9.3 Rolling windows and trend vs seasonality	23
9.4 Seasonality is why naive comparisons lie	24
Part 10 · LLM-Assisted Analysis	25
10.1 The three ways to use an LLM on data	25
10.2 Text-to-SQL, safely	25
10.3 Where LLMs genuinely help	26
Part 11 · Reproducible Analysis & Reporting	27
11.1 Reproducibility: raw data in, answer out, every time.	27
11.2 Notebooks vs scripts	27
11.3 Communicating the result	28
Part 12 · Capstone & Cheat Sheet	29
12.1 Capstone — one question, end to end	29
12.2 pandas ⇔ SQL cheat sheet	29
12.3 Analysis cheat sheet	29
12.4 The five rules.	30

How to Use This Book

Machine learning gets the attention, but under every model — and every honest business decision — is **data analysis**: the unglamorous, essential craft of loading messy data, cleaning it, exploring it, and turning it into an answer someone can act on. Analysts spend most of their time here, and so will you before any model is worth training. This book is that foundation, for a developer who already writes Python.

It's practical and tool-first. You'll live in **pandas** and **SQL**, reach for **statistics** when a number needs a caveat, use **visualization** to actually see what's going on, and — because it's 2026 — learn to use an **LLM as an analysis partner** (write the pandas for you, turn a question into SQL, draft the summary) without letting it fool you.

Each chapter answers four questions:

1. **What is this for?**
2. **How do you do it (pandas / SQL)?**
3. **What are the pitfalls?**
4. **When is a simpler tool the right call?**

The one idea: question → data → answer → decision

Good analysis is not "make a chart." It's a pipeline from a real question to a decision someone trusts:

DIAGRAM

```

QUESTION → get DATA → clean → explore/analyse → ANSWER → decision
"why did signups drop?"      (the 80% of the work)    with caveats
  
```

The skill isn't any single step — it's doing all of them honestly, especially resisting the urge to jump from raw data to a confident conclusion without the cleaning and the caveats in between.

ENGINEER'S MENTAL MODEL

Think of a DataFrame as an in-memory table and pandas as an ORM/query engine for it: `df[df.x > 3]` is a `WHERE`, `groupby().agg()` is `GROUP BY`, `merge()` is a `JOIN`. If you know SQL, you already know the *operations* — this book is about applying them to answer questions and about the judgement (cleaning, statistics, honesty) that turns query results into insight.

What this book covers

The analyst workflow; **pandas** (the core tool) and **analytical SQL** (where the data usually lives); **cleaning** messy real-world data; **reshaping** with group-by/pivot/join; **exploratory data analysis (EDA)**; the **statistics** that keep you honest (distributions, testing, correlation $\neq$ causation); **visualization**; a short **time-series** chapter (because "predict" often means forecasting); using **LLMs to analyse data**; and making the work **reproducible and reportable**.

HOW TO READ IT

Read Parts 1-6 in order — they're the core workflow and everything builds on pandas. Parts 7-9 (stats, viz, time series) are reach-for-when-needed. Part 10 (LLM analysis) assumes you've done the manual version first, on purpose — you can't supervise an LLM analyst if you've never done the analysis yourself. Do the exercises against a real CSV; this is a hands-on craft.

Part 1 · The Analyst's Workflow

Before tools, the shape of the work. Data analysis is a discipline with a repeatable process; skipping steps is how you get confident, wrong answers. This chapter is the map the rest of the book fills in.

1.1 The four kinds of analysis

The field labels analysis by the question it answers — a ladder of increasing ambition:

Kind	Question	Example	This book
Descriptive	What happened?	"signups fell 12% in March"	core (Parts 2–6, 8)
Diagnostic	Why did it happen?	"the drop was all on mobile"	core (Parts 5–7)
Predictive	What will happen?	"we'll lose ~9% next month"	forecasting (Part 9) → ML book
Prescriptive	What should we do?	"fix mobile checkout"	the decision (Part 11)

Most real analysis is descriptive and diagnostic — the "what" and "why." Those are the workhorses, and they're where careful pandas/SQL plus honest statistics earn their keep.

1.2 The pipeline, step by step

1. **Frame the question** — a vague ask ("look into signups") becomes a precise one ("did signup conversion drop, on which platform, and when?").
2. **Get the data** — from a database (SQL), a file (CSV/Parquet), or an API.
3. **Clean it** — missing values, wrong types, duplicates, outliers. The biggest time sink, and the most important (Part 4).
4. **Explore** — distributions, groupings, relationships (EDA, Part 6).
5. **Analyse** — compute the answer, with statistics to say how sure you are (Part 7).
6. **Communicate** — a chart and a sentence a decision-maker can act on (Parts 8, 11).

ENGINEER'S MENTAL MODEL

This is debugging, applied to a question instead of a bug. You form a hypothesis ("the drop is mobile"), pull the data, slice it, and let evidence confirm or kill the hypothesis — iterating fast. The discipline that stops you shipping a bad fix is the same one that stops you shipping a wrong conclusion: reproduce it, isolate it, and don't trust the first plausible story.

1.3 Frame the question first

The most common failure isn't technical — it's answering the wrong question. "Our numbers are down" could mean revenue, signups, or engagement; down versus last week, last month,

or last year; overall or in one segment. Nail the precise question, the metric, and the comparison *before* touching data, or you'll produce a beautiful analysis of the wrong thing.

COMMON MISTAKE

Diving into pandas before framing the question. You end up exploring aimlessly, producing charts nobody asked for, and anchoring on whatever pattern you happened to notice first. Write the question, the exact metric, and the comparison down in one sentence first — it turns an open-ended fishing trip into a focused investigation.

WHEN NOT TO DO A FULL ANALYSIS

If a stakeholder needs a single number they'll check weekly, don't hand-write a notebook each time — build a **dashboard** or a scheduled query. If the "analysis" is really "is A bigger than B," a quick query and a caveat beats a modelling exercise. Match the effort to the decision: a reversible, low-stakes call doesn't deserve a week of analysis.

EXERCISE 1.1

Take a vague question from your own work ("engagement is down," "which feature is popular"). Rewrite it as a precise, answerable question: the exact metric, the segment, and the comparison period. Then sketch — without code — which of the six pipeline steps it needs and where you expect the data to be messy.

Part 2 · pandas — Your Core Tool

pandas is where you'll spend most of your analysis time. It gives you the **DataFrame** — a labelled, in-memory table — and a huge vocabulary for slicing, filtering, and aggregating it. This chapter is the working core; later chapters apply it.

2.1 DataFrame and Series

A **DataFrame** is a table; a **Series** is one column (a labelled array). Load data and look at it before anything else:

PYTHON

```
import pandas as pd

df = pd.read_csv("signups.csv")      # also read_parquet, read_sql, read_json
df.head()                          # first rows – always look first
df.shape                            # (rows, columns)
df.info()                           # dtypes + non-null counts (spot problems)
df.describe()                       # summary stats for numeric columns
df["plan"].value_counts()           # frequency of each category
```

ENGINEER'S MENTAL MODEL

A DataFrame is a table you'd otherwise hold in SQL, and a Series is one column. `df.info()` is your `DESCRIBE` table plus a null audit — run it immediately on any new dataset. The mental shift from loops to **vectorised operations** (act on whole columns at once) is the one habit that makes pandas fast and readable; reaching for a `for` loop over rows is almost always the wrong move (2.4).

2.2 Selecting and filtering

PYTHON

```
df["email"]                         # one column (Series)
df[["email", "plan"]]               # several columns
df[df["age"] > 30]                  # filter rows (a WHERE)
df[(df["age"] > 30) & (df["plan"] == "pro")] # combine – note & and parentheses
df.loc[df["age"] > 30, "email"]     # filter rows AND pick columns
df.iloc[0:5]                       # by position
```

The `&` / `|` operators (not `and` / `or`) with **parentheses around each condition** is the single most common pandas syntax gotcha — get it into muscle memory.

2.3 Creating and transforming columns

PYTHON

```
df["revenue"] = df["price"] * df["quantity"]           # vectorised – whole column at
once
df["domain"] = df["email"].str.split("@").str[1]        # string methods via .str
df["signup"] = pd.to_datetime(df["signup"])            # proper datetime dtype
df["is_pro"] = df["plan"].eq("pro")                    # boolean column
df["band"] = pd.cut(df["age"], [0, 30, 50, 200],        # bin a number into categories
                      labels=["<30", "30-50", "50+"])

```

2.4 Vectorise — don't loop

PYTHON

```
# SLOW, un-pandastic:
df["total"] = [p * q for p, q in zip(df["price"], df["quantity"])]
# FAST, idiomatic:
df["total"] = df["price"] * df["quantity"]

```

COMMON MISTAKE

Iterating over rows (`for i, row in df.iterrows()`) to compute something. It's orders of magnitude slower and harder to read than a vectorised expression or, when you truly need row logic, a `df.apply(...)`. If you're writing a Python loop over a DataFrame, stop and ask "what's the column operation?" — there almost always is one.

PRODUCTION TIP

For data too big for memory, don't jump straight to Spark. First try **reading only the columns you need**, using efficient dtypes (categoricals, downcast numerics), reading **Parquet instead of CSV**, or **Polars/DuckDB** (both far faster and lower-memory than pandas on large files). Most "pandas is too slow / out of memory" problems are a file-format and dtype problem, not a need for a cluster.

EXERCISE 2.1

Load a CSV with at least 10k rows. Run `head`, `info`, `describe`, and `value_counts` on a categorical column. Create a derived numeric column vectorised, a datetime column with `to_datetime`, and a binned category with `cut`. Filter to a two-condition subset using `&` and parentheses. Time an `iterrows` version of one calculation against the vectorised version.

Part 3 · Analytical SQL

Most data lives in a database, and the fastest, cheapest place to do a lot of analysis is **in the database** with SQL — before a single row reaches pandas. As a developer you know CRUD SQL; *analytical* SQL adds a few power tools — aggregation, window functions, and CTEs — that do most analysis work.

3.1 Aggregation is the backbone

SQL

```
SELECT plan,
       COUNT(*)           AS users,
       AVG(revenue)       AS avg_rev,
       SUM(revenue)       AS total_rev
FROM subscriptions
WHERE created_at >= '2026-01-01'
GROUP BY plan
ORDER BY total_rev DESC;
```

`GROUP BY` + aggregates (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) answers most descriptive questions. `HAVING` filters *after* aggregation (the `WHERE` for groups).

3.2 Window functions — the analyst's superpower

Window functions compute across rows *related to the current row* without collapsing them — running totals, rankings, period-over-period, per-group row numbers. This is the single biggest step up from CRUD SQL:

SQL

```
SELECT user_id, month, revenue,
       SUM(revenue) OVER (PARTITION BY user_id ORDER BY month) AS running_total,
       revenue - LAG(revenue) OVER (PARTITION BY user_id ORDER BY month) AS
mom_change,
       RANK() OVER (PARTITION BY month ORDER BY revenue DESC) AS rank_in_month
FROM monthly_revenue;
```

- `SUM(...) OVER (...)` — running totals / moving windows.
- `LAG` / `LEAD` — reach the previous/next row (period-over-period change).
- `ROW_NUMBER` / `RANK` — top-N per group, dedup.

ENGINEER'S MENTAL MODEL

A window function is `GROUP BY` that *keeps the rows*. `GROUP BY` gives you one row per group; `OVER (PARTITION BY ...)` computes the same kind of aggregate but attaches it to every original row — like a correlated subquery you'd otherwise write by hand, but fast and readable. If you learn one new SQL thing for analysis, learn windows.

3.3 CTEs for readable, staged queries

Common Table Expressions (`WITH`) let you build an analysis in named steps instead of nested subqueries — the SQL equivalent of extracting well-named variables:

SQL

```
WITH monthly AS (
    SELECT date_trunc('month', created_at) AS month, COUNT(*) AS signups
    FROM users GROUP BY 1
),
with_growth AS (
    SELECT month, signups,
           signups - LAG(signups) OVER (ORDER BY month) AS growth
    FROM monthly
)
SELECT * FROM with_growth WHERE growth < 0;      -- months that shrank
```

3.4 Push work down to the database

Aggregate and filter in SQL, pull the *small* result into pandas for the finishing touches. Don't `SELECT *` a million rows into pandas to then group them — the database does that faster and moves far less data.

PYTHON

```
import pandas as pd
df = pd.read_sql("SELECT plan, COUNT(*) n FROM subs GROUP BY plan", conn)
```

COMMON MISTAKE

Pulling raw tables into pandas and doing all the aggregation there out of habit. You move gigabytes over the wire, blow up memory, and run slower than a `GROUP BY` the database was built for. Rule of thumb: **aggregate in SQL, refine in pandas**. Let the database do the heavy reduction; let pandas do the flexible last mile.

WHEN NOT TO USE SQL

If the data is already a modest file or the transformation is awkward to express in SQL (complex reshaping, row-wise Python logic, ML feature code), just do it in pandas. And for genuinely huge local files, **DuckDB** lets you run analytical SQL directly on CSV/Parquet with no database server — often the best of both worlds.

EXERCISE 3.1

Against any database (or DuckDB on a CSV), write: (1) a `GROUP BY` with two aggregates and a `HAVING` filter; (2) a window query with a running total and a `LAG`-based period-over-period change; (3) a CTE-staged query that finds groups which declined. Then reproduce query (1) in pandas and confirm the numbers match.

Part 4 · Cleaning Messy Data

This is where analysts spend most of their time, and where wrong answers are born. Real data has missing values, wrong types, duplicates, inconsistent categories, and outliers. Cleaning isn't glamorous, but every conclusion rests on it — a dirty dataset produces a confident, wrong result.

4.1 Find the problems first

PYTHON

```
df.info()                # dtypes + non-null counts
df.isna().sum()          # missing values per column
df.duplicated().sum()    # duplicate rows
df["plan"].value_counts() # spot "pro", "Pro", "PRO", " pro "
df.describe()            # impossible mins/maxes (age = -3, price = 1e9)
```

Always audit before you fix — you can't clean what you haven't found.

4.2 Missing values

PYTHON

```
df["age"].isna().mean()    # fraction missing — how bad is it?
df = df.dropna(subset=["email"]) # drop rows missing a critical field
df["age"] = df["age"].fillna(df["age"].median()) # impute a number (median > mean
if skewed)
df["plan"] = df["plan"].fillna("unknown") # explicit category for missing
```

The decision — drop, fill, or flag — depends on *why* it's missing and how much. There's no universal right answer, only a documented choice.

ENGINEER'S MENTAL MODEL

Missing data is like null-handling, but the danger is silent: pandas skips NaN in many operations, so a mean over a half-empty column returns a number that *looks* fine while quietly ignoring half your data. Decide missingness explicitly per column — drop, impute, or keep as a labelled category — the way you'd never let a null flow through business logic unchecked.

4.3 Types, duplicates, and inconsistent categories

PYTHON

```
df["price"] = pd.to_numeric(df["price"], errors="coerce")    # "12.50" -> 12.5; junk -
> NaN
df["signup"] = pd.to_datetime(df["signup"], errors="coerce")
df = df.drop_duplicates(subset=["user_id"])                # dedup on a key
df["plan"] = df["plan"].str.strip().str.lower()            # " Pro " -> "pro"
df["country"] = df["country"].replace({"USA": "US", "U.S.": "US"})
```

4.4 Outliers — investigate, don't reflexively delete

PYTHON

```
q1, q3 = df["amount"].quantile([0.25, 0.75])
iqr = q3 - q1
mask = (df["amount"] < q1 - 1.5*iqr) | (df["amount"] > q3 + 1.5*iqr)
df[mask]                                     # look at them before deciding
```

An outlier might be an error (a fat-fingered price) or the most important row in the dataset (the fraud, the whale customer). Look before you cut.

COMMON MISTAKE

Silently deleting outliers to make a chart look nice. Sometimes the outlier is the story — the fraudulent transaction, the one enterprise deal that's half your revenue. Removing it can erase the very thing worth finding. Investigate each outlier's cause; drop only genuine errors, and always report how many you removed and why.

PRODUCTION TIP

Keep cleaning **reproducible**: do it in a script/notebook that starts from the raw file every time, never by hand-editing the CSV. When someone questions a number (they will), you need to replay exactly how it was cleaned. A hand-edited spreadsheet is an un-reproducible, un-auditable result — treat raw data as read-only and express every fix as code.

WHEN NOT TO OVER-CLEAN

Don't polish columns you won't use, or impute a field that's 80% missing into a fake signal — dropping it is more honest. Clean what the question needs to the standard the decision needs, then stop. Perfect data is a myth; fit-for-purpose is the goal.

EXERCISE 4.1

Take a deliberately messy CSV (mixed-case categories, some missing ages, a few duplicate rows, a price column stored as strings with junk). Audit it with `info / isna / value_counts`, then clean each issue in code: coerce types, dedup on a key, normalise the category, decide-and-document your missing-value strategy, and investigate outliers with the IQR rule. Keep the raw file untouched.

Part 5 · Reshaping: Group-By, Pivot & Join

With clean data, most analysis is reshaping it to expose the answer: **split-apply-combine** (group-by), **pivoting** long↔wide, and **joining** datasets. These four verbs cover the bulk of diagnostic work — the "why" behind the "what."

5.1 Group-by: split-apply-combine

The workhorse. Split rows into groups, apply an aggregate, combine into a result — exactly SQL's `GROUP BY`:

PYTHON

```
df.groupby("plan")["revenue"].sum()           # one aggregate
df.groupby("plan")["revenue"].agg(["sum", "mean", "count"])
df.groupby(["plan", "country"])["revenue"].mean() # multi-level
df.groupby("plan").agg(                        # named, per-column
    aggregates
    users=("user_id", "nunique"),
    revenue=("revenue", "sum"),
    avg_age=("age", "mean"),
)
```

ENGINEER'S MENTAL MODEL

`groupby(...).agg(...)` is `GROUP BY ... SELECT agg(...)`. `transform` is the pandas answer to a window function — it returns a value *per original row* (e.g. each row's share of its group's total) instead of collapsing groups. If you catch yourself looping over unique values to aggregate each, that's a group-by.

5.2 Pivot: long ↔ wide

PYTHON

```
# long -> wide: months across the top, one row per plan
wide = df.pivot_table(index="plan", columns="month",
    values="revenue", aggfunc="sum", fill_value=0)
# wide -> long: back to tidy rows (great before plotting/modelling)
long = wide.reset_index().melt(id_vars="plan", var_name="month",
    value_name="revenue")
```

Tidy (long) data — one observation per row, one variable per column — is what plotting and modelling libraries want. **Wide** is what humans read in a table. You'll pivot between them constantly.

5.3 Join: combining datasets

PYTHON

```
merged = orders.merge(customers, on="customer_id", how="left")    # a LEFT JOIN
merged = a.merge(b, left_on="uid", right_on="id", how="inner")
```

The `how` (`left` / `inner` / `outer` / `right`) means exactly what it does in SQL. The classic bug is the join that silently changes your row count.

COMMON MISTAKE

Not checking row counts around a merge. If the right table has duplicate keys, a "lookup" join **multiplies** rows (a one-to-many becomes many rows), and your subsequent `sum` is silently inflated. Check `len(df)` before and after, verify join keys are unique where you expect them to be (`df["id"].is_unique`), and know whether each join is one-to-one or one-to-many *before* you run it.

PRODUCTION TIP

After any group-by or join, **sanity-check the totals**: does the sum of the grouped revenue equal the ungrouped total? Did the row count change the way you expected? A thirty-second reconciliation catches the majority of analysis bugs — a dropped category, a fan-out join, a filter you forgot — before they become a wrong number in a slide.

EXERCISE 5.1

From an orders table joined to a customers table: compute revenue per country with a group-by, pivot it to country × month wide form, then melt it back to tidy. Deliberately introduce a duplicate customer key, re-run the join, and watch the revenue total inflate — then fix it and reconcile the total against the ungrouped sum.

Part 6 · Exploratory Data Analysis (EDA)

EDA is the open-ended looking-around you do to understand a dataset before you commit to an answer — distributions, relationships, and surprises. It's where you form and kill hypotheses. Done well it prevents most wrong conclusions; skipped, it's how you confidently report an artifact of dirty data.

6.1 Look at one variable at a time (univariate)

PYTHON

```
df["age"].describe()           # center + spread
df["age"].hist(bins=30)        # the SHAPE – skew, gaps, multiple peaks
df["plan"].value_counts(normalize=True) # category proportions
df["amount"].quantile([.5, .9, .99]) # the tail (means hide it)
```

The **shape** matters more than the mean. A mean income of \\$80k means something very different if it's a tight bell curve versus a few billionaires dragging up a crowd at \\$30k. Always look at the distribution, not just the average.

6.2 Look at relationships (bivariate)

PYTHON

```
df.groupby("plan")["churned"].mean() # churn rate by plan (cat vs binary)
df.plot.scatter(x="tenure", y="revenue") # numeric vs numeric
df[["age", "tenure", "revenue"]].corr() # correlation matrix
pd.crosstab(df["plan"], df["country"]) # category vs category
```

6.3 Correlation is a hint, not a conclusion

A correlation says two things move together — nothing about *why*. It can be coincidence, reverse causation, or a hidden common cause (a **confounder**). More on this in Part 7; in EDA, treat every correlation as a **question to investigate**, not an answer.

ENGINEER'S MENTAL MODEL

EDA is exploratory logging for data: you don't yet know what's wrong, so you print everything — shapes, counts, cross-tabs — and let anomalies jump out. A weird spike, an impossible value, a bimodal histogram: these are your breakpoints. Follow each surprise before forming a story, the way you'd chase an unexpected log line before blaming the obvious code.

6.4 Segment — the averages lie

The single most valuable EDA habit: **break every aggregate down by segment**. An overall flat metric often hides a big rise in one group cancelling a big fall in another. "Signups are steady" can mean "up 40% on desktop, down 40% on mobile" — which is the actual story.

PYTHON

```
df.groupby(["month", "platform"])["signups"].sum().unstack() # trend per segment
```

COMMON MISTAKE

Reporting a single headline average and missing **Simpson's paradox** — where a trend that holds overall reverses within every subgroup (or vice versa). A treatment can look worse in aggregate yet be better for every individual group, purely due to how the groups are weighted. Always slice by the obvious segments (time, platform, cohort, region) before concluding anything from an aggregate.

WHEN NOT TO KEEP EXPLORING

EDA is open-ended, which makes it a great place to get lost. Once you've answered the framed question (Part 1) and sanity-checked it across the obvious segments, **stop** — further slicing risks finding a random pattern that won't replicate. If you slice enough ways, something will always look "significant" by chance (Part 7).

EXERCISE 6.1

On a real dataset: plot the distribution of one numeric variable and note its shape and tail. Compute a key metric (e.g. conversion) overall, then break it down by two segments and find a case where the segmented story differs from the headline. Build a correlation matrix and pick one strong correlation to argue — in a sentence — why it might *not* be causal.

Part 7 · The Statistics That Keep You Honest

You don't need a stats degree, but you need enough to attach the right caveat to a number — to say "signups rose 5%" *and* "but that's within normal noise." This chapter is the practical minimum that stops you from over-claiming.

7.1 Sampling and uncertainty

Almost every number you compute is from a **sample**, so it carries uncertainty. A conversion rate of 12% from 50 visitors is barely distinguishable from 8% or 16%; from 50,000 visitors it's solid. The tool for "how sure am I" is the **confidence interval** — a range the true value plausibly sits in.

PYTHON

```
import numpy as np
from scipy import stats
data = df["converted"].astype(float)          # 0/1 outcomes
mean = data.mean()
ci = stats.t.interval(0.95, len(data)-1, loc=mean, scale=stats.sem(data))
print(f"mean:.1%   95% CI [{ci[0]:.1%}, {ci[1]:.1%}]")
```

ENGINEER'S MENTAL MODEL

A point estimate with no interval is like a benchmark with no variance — a single number that hides whether it's stable or noise. "12%" invites a decision; "12% ± 6%" invites the right decision. Report the interval (or at least the sample size) whenever a number will drive a choice.

7.2 Hypothesis testing and the p-value

To ask "is this difference real or luck?", a hypothesis test gives a **p-value**: the probability of seeing a difference this big *if there were truly no difference*. Small p (conventionally < 0.05) → unlikely to be pure chance.

PYTHON

```
a = df[df.group=="A"]["converted"]
b = df[df.group=="B"]["converted"]
t, p = stats.ttest_ind(a, b)
print(f"p = {p:.3f}")          # small -> the groups probably really differ
```

A p-value is **not** the probability your hypothesis is true, and "not significant" is **not** "no effect" — it may just mean too little data.

7.3 Correlation vs causation

The cardinal rule. A correlation can arise from $A \rightarrow B$, $B \rightarrow A$, a **confounder** C causing both, or coincidence. To claim causation you need a **controlled experiment** (a randomised A/B test) or careful causal methods — never a correlation alone.

DIAGRAM

ice-cream sales ↔ drownings (both caused by: summer heat)
correlated, obviously not causal — the confounder is the real driver

7.4 A/B tests, done honestly

Randomised experiments are the clean way to establish causation. The traps are statistical: decide the sample size and duration **up front**, don't stop the moment it looks significant (**peeking** inflates false positives), and don't silently run twenty comparisons and report the one that "won" (**multiple comparisons** — with 20 tests at $p < 0.05$, one false positive is expected by chance).

COMMON MISTAKE

p-hacking, usually unintentional: slicing the data many ways, or checking an A/B test daily and stopping when it crosses 0.05. Test enough segments and something always looks significant by luck. Guard against it — pre-register the question and sample size, correct for multiple comparisons, and treat a surprising slice as a hypothesis to confirm on fresh data, not a finding.

WHEN NOT TO RUN A FORMAL TEST

If an effect is huge and obvious on a big sample (conversion doubled across 100k users), a formal test is a formality — report the effect size and move on. Statistics earns its keep on **small or ambiguous** differences, where noise and signal are easy to confuse. Effect size and practical significance usually matter more than the p-value anyway.

EXERCISE 7.1

Take an A/B dataset (or simulate one). Compute each group's conversion with a 95% confidence interval, run a t-test for the difference, and interpret the p-value in a sentence a non-statistician would understand. Then demonstrate p-hacking to yourself: split the data into 20 random segments, test each, and count how many come out "significant" by chance.

Part 8 · Visualization

A chart is how analysis becomes an answer someone acts on. The goal isn't a pretty picture — it's to make the finding *impossible to miss and hard to misread*. This chapter is about picking the right chart and not lying with it (even by accident).

8.1 The tools

- **matplotlib** — the foundation; total control, verbose.
- **seaborn** — statistical charts in one line on top of matplotlib; your default for exploration.
- **pandas .plot** — quick charts straight off a DataFrame.
- **plotly** — interactive charts for notebooks and the web.

PYTHON

```
import seaborn as sns
sns.histplot(df, x="age", bins=30) # distribution
sns.barplot(df, x="plan", y="revenue", errorbar="ci") # comparison + uncertainty
sns.lineplot(df, x="month", y="signups", hue="platform") # trend over time
sns.scatterplot(df, x="tenure", y="revenue") # relationship
```

8.2 Match the chart to the question

Question	Chart
Distribution of one variable	histogram / box plot
Compare a value across categories	bar chart
Trend over time	line chart
Relationship between two numbers	scatter plot
Composition (parts of a whole)	stacked bar (rarely pie)
Two categories' overlap	heatmap / crosstab

ENGINEER'S MENTAL MODEL

Choosing a chart is choosing the right data structure for a job: a line implies continuity (time), a bar implies discrete comparison, a scatter implies a relationship. Using the wrong one misleads even with correct numbers — a line chart across unordered categories invents a trend that isn't there. Pick the encoding that matches what the data is.

8.3 Don't mislead (even accidentally)

- **Bar charts start at zero.** Truncating the y-axis turns a 2% difference into a cliff. (Line charts of a trend can be an exception.)
- **Label axes and units.** An unlabelled axis is a guess.
- **Don't over-plot.** 50 series is noise; highlight the few that matter.
- **Avoid pie charts** for more than 2–3 slices — humans read angles badly; a bar chart is almost always clearer.

COMMON MISTAKE

A truncated y-axis that exaggerates a tiny difference — the most common way honest people mislead with data. A bar chart from 98 to 100 makes a 2% difference look enormous. Start bars at zero, and be suspicious of any chart (yours or others') where the axis doesn't. The chart should make the *real* magnitude obvious, not inflate it.

PRODUCTION TIP

Every chart that leaves your notebook needs a **title that states the finding** (not "Revenue by month" but "Revenue fell 12% after the March pricing change") and labelled axes with units. The reader should get the point from the title and confirm it from the chart — not have to reverse-engineer what you meant. The title is the sentence; the chart is the evidence.

WHEN NOT TO MAKE A CHART

A single number (one KPI, one percentage) is clearer as **text** than as a lonely bar. And if a chart needs a paragraph to explain, it's the wrong chart — simplify it or say it in a sentence. Not everything needs a visualization; the goal is the fastest path to understanding.

EXERCISE 8.1

For one finding in your data, make the *right* chart (match it to the question from 8.2), give it a finding-stating title and labelled axes, and start any bar chart at zero. Then deliberately make a misleading version (truncated axis, or a pie with eight slices) and put them side by side — internalise how easy the distortion is.

Part 9 · Time-Series Analysis

A huge share of real questions are about data over time — signups per day, revenue per month, traffic per hour. Time series has its own tools and its own traps, and it's the bridge to *predictive* analysis (forecasting) that the ML book picks up.

9.1 A datetime index unlocks pandas' time tools

PYTHON

```
df["ts"] = pd.to_datetime(df["ts"])
df = df.set_index("ts").sort_index()           # a DatetimeIndex enables the rest
```

9.2 Resampling — change the time grain

PYTHON

```
daily    = df["amount"].resample("D").sum()      # to daily totals
monthly  = df["amount"].resample("ME").sum()     # to month-end totals
weekly   = df["amount"].resample("W").mean()     # weekly averages
```

`resample` is `groupby` for time — bucket rows into periods and aggregate. It's how you go from raw events to "per day/week/month."

9.3 Rolling windows and trend vs seasonality

PYTHON

```
monthly.rolling(3).mean()                    # 3-month moving average (smooths
noise)
monthly.pct_change()                         # month-over-month growth
monthly.pct_change(12)                       # vs same month last year (YoY)
```

Two ideas run through all time series:

- **Trend** — the long-run direction (are we growing?).
- **Seasonality** — repeating patterns (weekends dip, December spikes).

A moving average reveals the trend by smoothing the noise; comparing to the same period last year (`pct_change(12)`) removes seasonality so you see real change.

ENGINEER'S MENTAL MODEL

Resampling is a `GROUP BY date_trunc(...)` with a proper time index, and a rolling window is a SQL window function over time. The new discipline is respecting **order and gaps**: rows aren't independent, missing days aren't zero unless you say so (`resample + fillna(0)`), and you must never shuffle the sequence.

9.4 Seasonality is why naive comparisons lie

"Sales dropped 30% since last week" might just mean last week contained a holiday, or this week is a weekend. Compare **like periods** — this month vs the same month last year, this Tuesday vs typical Tuesdays — before declaring a change.

COMMON MISTAKE

Reading a weekly or monthly wiggle as a trend when it's seasonality or a calendar artifact (a month with fewer business days, a holiday). Always de-seasonalise (year-over-year, or a moving average over a full seasonal cycle) before claiming "growth" or "decline." Most "the metric crashed!" panics are seasonality plus a short comparison window.

WHEN NOT TO FORECAST

Extrapolating a trend into the future is *prediction* — a different, harder discipline (the ML book: Prophet, sktime, ARIMA). Describing what happened (this chapter) is safe; forecasting what will happen invites overconfidence, especially near turning points. Don't quietly extend a line off the edge of the chart and call it analysis — that's a model, and it needs a model's caveats.

EXERCISE 9.1

Take an events dataset with timestamps. Set a datetime index, resample to daily and monthly totals, and plot both. Add a 7-day moving average to expose the trend under the daily noise. Compute month-over-month and year-over-year growth and find a case where the MoM number is misleading but the YoY number tells the real story.

Part 10 · LLM-Assisted Analysis

An LLM is a genuinely useful analysis partner — it writes pandas and SQL for you, turns a plain-English question into a query, and drafts the summary. It's also a confident liar about numbers. This chapter is how to use it to go faster *without* letting it produce authoritative-sounding nonsense.

10.1 The three ways to use an LLM on data

1. **Code assistant** — "write pandas to compute monthly churn by plan." You get the code, you run it, you see the real numbers. Safest and most useful.
2. **Text-to-SQL** — the user asks in English, the LLM writes SQL, you run it against the database. Powers "chat with your data" features.
3. **Analyst agent** — an LLM with a tool that executes code/SQL, so it can load a file, explore, and answer iteratively (the agent loop from the LLM book).

ENGINEER'S MENTAL MODEL

The reliable pattern is **LLM writes code, your machine runs it**. The model is great at the *syntax* ("how do I express this group-by?") and unreliable at the *arithmetic* ("what's the actual number?"). Keep the deterministic execution on your side — pandas/SQL compute the truth — and use the LLM for the translation and the narration around it. Never let the model *report a figure it didn't compute by running code*.

10.2 Text-to-SQL, safely

DIAGRAM

user question → LLM (given the schema) → SQL → YOUR db runs it → real rows → answer

Give the model the **schema** (table and column names, types) so it writes correct queries. Then guard the execution — because you're running model-generated SQL:

- **Read-only** database role. The model should never be able to `DELETE / UPDATE`.
- **Validate/limit** the query (allow only `SELECT`, add a `LIMIT`, time it out).
- **Show the SQL** to the user so a wrong query is visible, not hidden behind a confident sentence.

COMMON MISTAKE

Trusting a number the LLM *states* rather than one it *computed by running code*. Ask a model "what's the average order value?" over pasted data and it will produce a plausible, specific, often **wrong** figure — it pattern-matched, it didn't calculate. Only trust numbers that came out of executed pandas/SQL. The LLM writes the query; the database owns the answer.

10.3 Where LLMs genuinely help

Boilerplate pandas/SQL, remembering syntax you use rarely (regex, window functions, date math), explaining someone else's query, suggesting what to look at next, and drafting the plain-English summary of results *you've verified*. Used this way it's a real speed-up on the tedious parts.

PRODUCTION TIP

For a "chat with your data" feature, the trustworthy architecture is: **schema-in → SQL-out → read-only execution → return the rows and the SQL**, with the model only *narrating* results the database actually returned. That's an agent (LLM book) with a single, sandboxed, read-only query tool. Everything dangerous — write access, unlimited scans, hidden math — is designed out.

WHEN NOT TO REACH FOR THE LLM

For a number that will drive a real decision or go in a report, do (or at least verify) the analysis yourself — the LLM's confidence is uncorrelated with its correctness on arithmetic. And you can't supervise what you can't do: if you've never written the group-by by hand, you can't tell when the model's version is subtly wrong. Learn the manual craft (Parts 2–9) first; then let the LLM accelerate it.

EXERCISE 10.1

Build a tiny text-to-SQL helper: give an LLM your table schema and a plain-English question, have it return **only** SQL, run that SQL against a read-only connection (with a `LIMIT`), and print both the query and the result. Then ask the same model to *state* the answer from pasted data without running code, and compare — see for yourself how often the un-executed number is wrong.

Part 11 · Reproducible Analysis & Reporting

An analysis nobody can reproduce — or understand — is worthless, however clever. This chapter is the engineering discipline that turns a notebook full of exploration into a result a team can trust, rerun, and act on. It's where your developer instincts are an advantage over many analysts.

11.1 Reproducibility: raw data in, answer out, every time

The rule: **anyone should be able to rerun your analysis from the raw data and get the same numbers.** That means:

- **Raw data is read-only.** Never hand-edit the source file; every fix is code (Part 4).
- **Pin the inputs** — the exact query or file, with a date. "The users table" isn't reproducible; "the users table as of 2026-03-01, this query" is.
- **Version the analysis** — it's code; commit it. A notebook or script in Git, not a one-off in someone's head.
- **Pin the environment** — a `requirements.txt` / `uv.lock` so the libraries match.

ENGINEER'S MENTAL MODEL

This is just reproducible builds applied to analysis: raw data = source, cleaning + analysis code = the build, the numbers = the artifact. If the build isn't deterministic and version-controlled, you can't debug a disputed figure or trust it next quarter. You already have these instincts — most analysts don't; use them.

11.2 Notebooks vs scripts

- **Notebooks** are for *exploration* and *communication* — interleaving code, output, and narrative. Perfect for EDA and for sharing a finding.
- **Scripts/pipelines** are for *repeated* analysis — a daily metric, a report that regenerates. When a notebook becomes something you run again and again, promote its logic to a script (or a scheduled job) so it's testable and automatable.

PRODUCTION TIP

Restart-and-run-all before you trust or share a notebook. Out-of-order execution means the numbers on screen may have come from code you've since changed or deleted — a notebook that only works because of hidden state is not reproducible. A clean top-to-bottom run is the notebook equivalent of a green CI build.

11.3 Communicating the result

The analysis isn't done when you have the number — it's done when someone *understands and can act on it*. Lead with the answer, support it with the evidence, and be honest about the limits:

- **The finding, in one sentence** — the headline, up front (the "so what").
- **The evidence** — the key chart or table (Part 8), not every chart you made.
- **The caveats** — sample size, confidence, what you assumed, what you cleaned (Parts 4, 7). The caveats are what make you trustworthy.
- **The recommendation** — what you'd do about it, if asked.

COMMON MISTAKE

Presenting the *journey* instead of the *answer* — twelve exploratory charts in the order you happened to make them, with the finding buried on slide nine. Decision-makers want the conclusion first. Do the messy exploration privately, then report the answer, the one chart that proves it, and the caveats. Nobody needs to see every dead end.

WHEN NOT TO BUILD A PIPELINE

A genuinely one-off question ("what happened last Tuesday?") doesn't need a productionised pipeline, tests, and a scheduler — a clean notebook run and a clear write-up is the right amount of engineering. Reserve the pipeline for analyses that *repeat*. Over-engineering a one-off is as wasteful as under-engineering a recurring report.

EXERCISE 11.1

Take one analysis you've done in this book and make it fully reproducible: read-only raw data, pinned query/file with a date, all cleaning in code, committed to Git with a pinned environment, and a restart-and-run-all that reproduces every number. Then write the result up in four parts — finding, evidence, caveats, recommendation — with the headline first.

Part 12 · Capstone & Cheat Sheet

12.1 Capstone — one question, end to end

Pick a real dataset and a real question, and run the whole pipeline:

1. **Frame** the precise question, metric, and comparison (Part 1).
2. **Get** the data — a SQL query (aggregate in the database) or a file (Part 3).
3. **Clean** it in code from the read-only raw source; document every choice (Part 4).
4. **Reshape** with group-by / pivot / join, reconciling totals (Part 5).
5. **Explore**, slicing by the obvious segments; chase the surprises (Part 6).
6. **Analyse** with a confidence interval or test; separate correlation from causation (Part 7).
7. **Visualize** the finding with the right chart, a finding-stating title, and a zero-based axis (Part 8); handle time carefully if it's a trend (Part 9).
8. **Report** — finding first, evidence, caveats, recommendation — from a reproducible, committed notebook (Part 11).

Optionally, wire an LLM text-to-SQL helper over the same data (Part 10) and verify its numbers against your own.

12.2 pandas ⇔ SQL cheat sheet

DIAGRAM

<code>df[df.x > 3]</code>	<code>WHERE x > 3</code>
<code>df[["a", "b"]]</code>	<code>SELECT a, b</code>
<code>df.groupby("k")["v"].sum()</code>	<code>SELECT k, SUM(v) ... GROUP BY k</code>
<code>df.groupby("k").agg(n=("id", "count"))</code>	<code>SELECT k, COUNT(id) ... GROUP BY k</code>
<code>df.merge(o, on="id", how="left")</code>	<code>LEFT JOIN o USING (id)</code>
<code>df.sort_values("v", ascending=False)</code>	<code>ORDER BY v DESC</code>
<code>df.drop_duplicates("id")</code>	<code>SELECT DISTINCT ON (id) ...</code>
<code>df["v"].transform(...) per group</code>	<code>SUM(v) OVER (PARTITION BY ...)</code>
<code>df.resample("ME").sum()</code>	<code>GROUP BY date_trunc('month', ts)</code>

12.3 Analysis cheat sheet

DIAGRAM

Always first:	<code>df.head(), df.info(), df.isna().sum(), df.describe()</code>
Filter safely:	<code>df[(a) & (b)]</code> — & / and parentheses, not and/or
Aggregate:	<code>groupby().agg(named=(col, func))</code>
Reshape:	<code>pivot_table (long→wide), melt (wide→long)</code>
Join check:	<code>len()</code> before/after; is the key unique?
Distribution:	look at the SHAPE, not just the mean; check the tail
Segment:	break every average down by time / platform / cohort
Uncertainty:	report a CI or the sample size, not a bare point
Causation:	correlation ≠ cause; needs an experiment
Time:	compare like periods (YoY), de-seasonalise before "trend"

12.4 The five rules

1. **Frame the question first** — the metric, the segment, the comparison — before any code.
2. **Cleaning is the job.** Raw data is read-only; every fix is reproducible code.
3. **Averages lie; segment everything.** Watch shapes and tails, not just means.
4. **Attach a caveat to every number** — a confidence interval or sample size — and never confuse correlation with causation.
5. **The LLM writes the query; the database owns the answer.** Only trust numbers that were computed by running code.

YOU'RE READY

You can now take a vague question to a trustworthy, reproducible answer: pandas and SQL to wrangle it, cleaning and statistics to keep it honest, visualization to make it land, and an LLM to go faster without being fooled. This is the foundation the Machine Learning and RL books build on — every model starts as clean, well-understood data. Go answer something real.